GUIDE OFFICIEL DE SOUTENANCE (3 ÉTUDIANTS)

Projet : Qualification & Validation Logicielle de la Plateforme gestionAlumni

ENICarthage (2025 – 2026)
Module : Qualité & Test Logiciel
Enseignant : **M. Faouzi Jaidi**

⚡ AVANT L'ARRIVÉE DU PROFESSEUR (MISE EN ROUTE INDISPENSABLE)

Ouvrez **2 terminaux discrets en arrière-plan** afin que toutes les démos live répondent en moins d'une seconde :

Terminal 1 (Backend 8081) :

```
cd C:\Users\LENOVO\Desktop\spring\gestionAlumni
set JAVA_HOME=C:\Users\LENOVO\Desktop\jdk-17.0.12
set PATH=C:\Users\LENOVO\Desktop\jdk-17.0.12\bin;%PATH%
mvnw.cmd spring-boot:run
```

Terminal 2 (Frontend 4200) :

```
npx serve -s "C:\Users\LENOVO\Desktop\spring\ProjetWeb\dist\demo-web\browser" -l 4200
```

🥷 ÉTUDIANT 1 : Chef de Projet & Backend Core

BASE DE LA PYRAMIDE

Périmètre : Logique métier des 9 services, base de la pyramide (ISTQB), tests d'isolation avec Mockito, base in-memory H2, contrôleurs REST MockMvc, et métriques JaCoCo.

DISCOURS D'INTRODUCTION (À DIRE AU PROFESSEUR) :

« Monsieur, nous avons appliqué les recommandations de l'ISTQB en consolidant la base de notre pyramide de tests avec 66 tests automatisés. Nous avons découplé nos services avec des mocks Mockito et utilisé une base H2 en mémoire pour garantir des tests hermétiques, indépendants et déterministes. Voici l'exécution en direct. »

COMMANDES À LANCER EN DIRECT DANS LE TERMINAL :

```
cd C:\Users\LENOVO\Desktop\spring\gestionAlumni
set JAVA_HOME=C:\Users\LENOVO\Desktop\jdk-17.0.12
set PATH=C:\Users\LENOVO\Desktop\jdk-17.0.12\bin;%PATH%
mvnw.cmd test
```

OUVRIR LE RAPPORT DE COUVERTURE JACOCO DANS CHROME :

```
start target\site\jacoco\index.html
```

Ce qu'il montre à l'écran & Arguments clés :

- Pointer le résultat : **Tests run: 66, Failures: 0, Errors: 0** (100% de succès).
- Montrer le tableau JaCoCo : Couverture **> 80 %** sur les services métiers clés.
- Mentionner les 22 tests d'intégration MockMvc validant les contrats HTTP sans dépendance externe.

🥷 ÉTUDIANT 2 : Qualité Statique, Sécurité & Contrats d'API

TECHNIQUES STATIQUES & API

Périmètre : Analyse statique SpotBugs, Revue formelle selon la norme **IEEE 1028**, validation des 13 requêtes d'API REST avec Postman/Newman, et sécurité d'accès.

DISCOURS D'INTRODUCTION (À DIRE AU PROFESSEUR) :

« Pour les techniques statiques, nous avons combiné une analyse outillée avec SpotBugs et une inspection formelle rigoureuse selon la norme IEEE 1028 avec modérateur, lecteur et rapporteur. De plus, nous avons testé nos contrats d'API avec Postman pour vérifier les codes HTTP, les payloads JSON et les temps de réponse. »

1. LANCER LE SCAN SPOTBUGS EN DIRECT :

```
mvnw.cmd spotbugs:spotbugs
```

2. LANCER LA SUITE D'API POSTMAN EN DIRECT AVEC NEWMAN :

```
cd C:\Users\LENOVO\Desktop\spring\gestionAlumni\tests\postman
npx newman run gestionAlumni.postman_collection.json -e gestionAlumni_environment.json
```

Ce qu'il montre à l'écran & Arguments clés :

- **SpotBugs :** 42 classes compilées analysées, **0 anomalie bloquante** détectée.
- **Postman / Newman :** 13 requêtes exécutées, **28 assertions validées**, latence moyenne 73 ms.
- **Explication du Cold Start :** Si TC-API-01 prend 2s au tout premier run, expliquer : *« C'est le démarrage à froid de la JVM et l'ouverture du pool HikariCP. Dès la 2e requête, la latence chute sous 100 ms. »*
- **Inspection IEEE 1028 :** Ouvrir le tableau 4.4 de votre rapport et présenter les défauts relevés (D-01 absence de RBAC, D-03 contournement service) pour prouver la démarche critique.

🥷 ÉTUDIANT 3 : Performance (JMeter), Interface (Selenium) & TestRail

NON-FONCTIONNEL & UI

Périmètre : Montée en charge (BNF01), tests E2E Selenium (NF-06), audit de qualité Google Lighthouse (NF-07), et centralisation des exigences dans TestRail.

DISCOURS D'INTRODUCTION (À DIRE AU PROFESSEUR) :

« Pour valider nos exigences non fonctionnelles, nous avons soumis notre API à des tirs de charge concurrents pour valider BNF01 (< 500ms). Ensuite, nous avons automatisé des tests E2E sous Selenium avec Chrome Headless pour valider la compatibilité multi-écrans NF-06, audité l'interface avec Google Lighthouse, et centralisé le tout dans TestRail. »

1. LANCER LE TEST DE MONTÉE EN CHARGE EN DIRECT (EXIGENCE BNF01) :

```
npx autocannon -c 10 -d 5 http://localhost:8081/api/alumni/directory
```

2. LANCER LES 8 TESTS SELENIUM E2E EN DIRECT :

```
cd C:\Users\LENOVO\Desktop\spring\gestionAlumni
pytest tests\selenium\test_interface_angular.py -v
```

3. OUVRIR LIGHTHOUSE ET LE DASHBOARD TESTRAIL DANS CHROME :

```
start preuves_rapport\05_interface_angular\lighthouse_accueil_desktop.report.html
start tests\testrail\testrail_dashboard_simulation.html
```

Ce qu'il montre à l'écran & Arguments clés :

- **Performance (Autocannon / JMeter) :** 284 ms de latence médiane, 0 % d'erreur HTTP sous 10 VUs.
- **Selenium E2E :** 8 tests réussis (100% Passed) validant les formats Mobile (375px), Tablette et Desktop.
- **Google Lighthouse :** Performance Desktop 94 %, Accessibilité 92 %, Bonnes Pratiques 100 %.
- **TestRail :** Centralisation de la campagne, traçabilité des exigences BF01-BF12 et taux global de 94 %.

TABLEAU RÉCAPITULATIF POUR LE JURY

Membre	Périmètre & Chapitres du Rapport	Outils Mobilisés	Démonstration Live	Normes Clés
Étudiant 1	Logique métier, TU & Intégration (Chap. 3 & 5.1)	JUnit 5, Mockito, Spring Boot, H2, JaCoCo	`mvnw.cmd test` + JaCoCo HTML	ISTQB (Boîte blanche, branches)
Étudiant 2	Techniques statiques & API (Chap. 4 & 5.2)	SpotBugs, IEEE 1028, Postman, Newman	`spotbugs` + `newman run`	IEEE 1028 (Inspection formelle)
Étudiant 3	Performance, E2E & Qualité UI (Chap. 5.3, 5.4, 2.3)	JMeter, Selenium, Lighthouse, TestRail	`autocannon` + `pytest` + HTML	ISO 25010 (Qualité logicielle)

💡 **CONSEILS EN OR POUR LA SOUTENANCE :**

1. **Ne lisez pas de diapositives chargées** : montrez les commandes qui tournent en direct dans le terminal, les profs adorent voir du concret.
2. **Employez le vocabulaire exact** : *« déterministe », « isolation par mocks », « rollback transactionnel », « cold start JVM », « traçabilité bidirectionnelle »*.
3. **Appuyez-vous sur votre rapport officiel** : Toutes les figures et tableaux (TAB 1.1 à TAB 6.2) y sont consignés !